Relatório técnico — em validação

Lumina Odonto | Sprint 7 | Docker, banco de dados e backend clínico

- **Data da consolidação:** 29/10/2026
- **Disciplinas:** Fábrica de Software e Extensão, conforme o README do projeto.
- **Período documentado:** planejamento de 19 a 24/10/2026; integrações e análise até 29/10/2026.
- **Branch examinada:** `chore/sprint-07-docker-integration`
- **HEAD local:** `2f7dbfe6d0d7a747f57dbeeede3a716938adc5fa`
- **Situação:** recorte técnico preparado para revisão de Paulo Sérgio e repasse a Cassiano Augusto. Não é aceite de QA nem declaração de conclusão integral da Sprint 7.

Sumário

1. Identificação, escopo e resumo executivo
2. Contexto e evolução da branch
3. Arquitetura integrada e execução
4. Banco de dados, migrações e recuperação
5. Backend e contratos HTTP
6. Requisitos, regras e estado de aprovação
7. Segurança, transações e concorrência
8. Testes e evidências disponíveis
9. Integração pendente e relação com o relatório cumulativo
10. Pendências, espaço de QA e conclusão
11. Fontes e rastreabilidade

1. Identificação, escopo e resumo executivo

Este relatório reúne o recorte de infraestrutura Docker, esquema PostgreSQL e API clínica da Sprint 7. Baseia-se nos relatórios de DBA e backend entregues por Paulo em 29/10, no histórico Git local, nos arquivos da branch e nos repasses técnicos versionados. Os dois relatos foram confrontados com código e documentação; uma contagem repetida nos relatos representa **um mesmo ensaio**, não dois. Os oito documentos de requisitos e regras alterados no checkout em 29/10 são **revisão local sem commit** e não pertencem ao HEAD indicado acima.

O problema inicial era consolidar a interface clínica herdada da Sprint 6, restabelecer a inicialização da API e executar frontend, API e PostgreSQL em conjunto. Depois, a equipe precisou substituir a leitura clínica ampla por autorização vinculada a evidências confiáveis de confirmação e atendimento. O planejamento da Sprint 7 inclui ainda dashboard de consultas, relatórios, pesquisa, filtros, exportação e histórico; a existência dessas metas não comprova sua entrega.

Entregas encontradas no código: Compose com PostgreSQL 16, FastAPI e frontend React/Vite; esquema e duas migrações SQL de autorização clínica; serviços da Agenda que registram confirmação e conclusão com evidências; leitura clínica que consulta o fundamento assistencial; cinco rotas de prontuário/evoluções e seis rotas de Agenda

documentadas. **Validação relatada:** integração Docker básica em 23/10, ensaios SQL em PostgreSQL 16.15 descartável em 24 e 25/10, 82 testes SQLite e cinco testes HTTP PostgreSQL em 26/10. Esses resultados pertencem às versões e ambientes indicados na seção 8; não foram repetidos nesta elaboração.

Situação atual: o frontend ainda envia apenas `status` ao concluir e oferece ações incompatíveis com a API; o GET do prontuário ainda cria dados; duas lacunas concorrentes permanecem; o banco persistente de revisão não tem migração clínica comprovada; falta aceite de João na versão integrada efetivamente executada. O código disponível, uma imagem construída e um contêiner atualmente em execução são fatos distintos. Não houve inspeção de imagens, contêineres ou banco nesta tarefa.

2. Contexto e evolução da branch

A Sprint 6 deixou a interface clínica e o fluxo de Agenda/prontuário que a Sprint 7 recebeu. O registro da integração Docker atribui a infraestrutura inicialmente preparada a Luiz Otávio, a partir de `e293d3c`, e documenta sua incorporação à base funcional após correções de inicialização. A tabela abaixo usa as **datas dos commits** do Git local; decisões e ensaios têm datas próprias, descritas depois. A presença na branch não confirma execução da versão em contêiner nem aceite acadêmico.

Commit e data	Entrega	Motivação	Arquivos principais	Evidência disponível
`34b128f` · 18/10	Interface clínica herdada da Sprint 6	Trazer Agenda e prontuário à base de integração	`frontend/src/App.tsx`, `frontend/src/api.ts`	Código versionado; fluxo atual de conclusão ainda diverge da API.
`4fec7ca` · 19/10	Plano da Sprint 7	Organizar manutenção, dashboard e responsáveis	`docs/sprint/sprint-07.md`	Planejamento, não resultado de execução.
`ab129d7` · 21/10	Correção de inicialização	Restaurar importação e mapeamentos da API	`backend/app/main.py`, modelos e teste de health	Código e teste de inicialização no histórico.
`9b26c81` · 23/10	Compose PostgreSQL/API/frontend	Reaproveitar e integrar a infraestrutura de Otávio	`docker-compose.yml`, Dockerfiles, Nginx	Ensaio isolado de três serviços e persistência básica, registrado em 23/10.
`698a3bd` · 24/10	Requisitos clínicos documentados	Formalizar a política de acesso então vigente	RF-023, RN-031 e índices	Documento histórico; política substituída pela revisão de 25/10.
`01a0157` · 24/10	Primeira concessão persistente	Guardar consulta de origem e preparar migração	`schema.sql`, migração de 24/10, verificadores SQL	Ensaio SQL em dois bancos descartáveis em 24/10.
`be7c2f2` · 26/10	Evidências de confirmação e realização	Adequar o SQL à decisão de 25/10	`schema.sql`, migração de 25/10, scripts	Três caminhos SQL ensaiados em 25/10; commit em 26/10.
`ac180a7` · 26/10	Integração da API com evidências	Aplicar transições e fundamento de leitura clínica	Agenda, `records/access.py`, testes	Código e testes incluídos; aceite integral pendente.
`ef15a63` · 26/10	Proteção parcial de escritas concorrentes	Bloqueios, rollback e testes de serviço/HTTP	Serviço clínico, testes, repasse técnico	82 SQLite e cinco PostgreSQL relatados para essa etapa.
`2f7dbfe` · 28/10	Guias de contratos HTTP	Registrar rotas e limites observados	READMEs de Agenda e prontuário	Guias versionados; não indicam nova execução da interface.

Cronologia distinta: a política de 24/10 mantinha leitura após mudança de estado apenas pela confirmação. Paulo a revisou em **25/10**: preparação exige confirmação confiável e consulta atualmente `CONFIRMED`; continuidade exige atendimento realizado com evidência válida. O ensaio SQL da política nova ocorreu em **25/10**, e seu commit foi criado em **26/10**. Os ensaios HTTP relatados são de **26/10**. O prazo de **24/10** consta do plano acadêmico; o repasse a Cassiano em **30/10** é meta proposta, sem fonte de prorrogação acadêmica aprovada. O estado remoto do PR não foi consultado nesta elaboração; a referência local `origin/chore/sprint-07-docker-integration` coincide com o HEAD.

3. Arquitetura integrada e execução

O Compose define três serviços na mesma rede: PostgreSQL `16-alpine`, API FastAPI/Uvicorn e frontend React/Vite servido por Nginx. O Dockerfile da API constrói a imagem a partir de Python 3.12, instala o projeto e inicia Uvicorn sob usuário sem privilégio. O Dockerfile do frontend usa Node 22 para `npm ci` e build Vite, depois publica os arquivos estáticos em Nginx 1.27. `VITE_API_URL` entra **no build**; mudança de URL ou código exige nova imagem de frontend.

O Compose passa `DATABASE_URL` à API apontando para `db:5432`, aguarda `pg_isready` do banco e usa health checks próprios para API e frontend. O endpoint `/health` indica que o processo HTTP responde; sozinho, não verifica esquema, consulta SQL ou autorização clínica. O volume nomeado `lumina_odonto_postgres_data` preserva dados entre reinícios. `schema.sql` é montado no diretório de inicialização do PostgreSQL e roda somente quando o volume está vazio. O Compose não aplica automaticamente Alembic nem migrações incrementais a bancos existentes.

O registro de 23/10 documenta build e execução isolada dos três serviços, `/health` 200, conexão da API ao PostgreSQL, login relatado e 50 pacientes fictícios recuperados após reinício. Esse ensaio antecede os commits de evidência e API de 26/10. Não há, neste recorte, IDs de imagens, comando de build e fluxo pela interface que demonstrem que o HEAD `2f7dbfe` esteja atualmente em execução. As variáveis reais de ambiente, senhas e tokens não são reproduzidos aqui.

4. Banco de dados, migrações e recuperação

4.1 Estrutura e integridade

Entidade	Estrutura relevante	Restrição e alcance da prova
Agenda `appointments`	Clínica, paciente, dentista, `scheduled_at` com fuso, situação, duração e período calculado	FKs e CHECKs de situação/duração; exclusão GiST impede sobreposição ativa do mesmo dentista. Chave composta `(id, clinic_id, patient_id, dentist_id)` sustenta a origem clínica.
`medical_records`	Prontuário por paciente, observações gerais e horários	`patient_id` único: uma linha por paciente; presença do número cadastral não prova existência do prontuário clínico.
`clinical_evolutions`	Prontuário, autor, consulta opcional, descrição, rascunho/finalizada e horários	Consulta vinculada é única; CHECK de estado/finalização não torna a linha finalizada imutável por si só.
`clinical_access_grants`	Uma linha por consulta; clínica, paciente, dentista, origem e `granted_at`; confirmador e três campos `care_*`	FK composta preserva participantes da origem; CHECK exige trio de realização completo e confirmador identificado. Índice `(clinic_id, dentist_id, patient_id)` atende à busca.
`audit_logs`	Estrutura geral prevista no SQL	Tabela existente não comprova auditoria funcional de confirmação, conclusão ou retificação.

A migração de 24/10 cria a concessão inicial. A migração de 25/10 acrescenta `confirmed_by_user_id`, `care_occurred_at`, `care_recorded_at` e `care_recorded_by_user_id`. Os quatro campos são anuláveis para preservar linhas antigas; uma linha legada sem confirmador identificado não sustenta, por isso, acesso confiável. A FK composta restringe troca comum dos participantes depois da concessão. A política para correção excepcional dessa situação ainda depende de Paulo. O SQL não demonstra por si mesmo qual ator autenticado confirmou ou concluiu: isso depende da API e de sua transação.

4.2 Destinos de banco

Origem	Caminho documentado	Limite
Banco novo e vazio	`schema.sql` atual, uma vez, no primeiro provisionamento	Não prova migração de dados preexistentes.
Esquema SQL completo anterior a 24/10	Migração de 24/10 e depois a de 25/10, após inventário e ensaio em restauração	Migrações não são reaplicáveis; exigem PostgreSQL 16.
Esquema de 24/10 já presente	Somente migração de 25/10 após conferência do catálogo	Linhas antigas mantêm novos campos nulos.
Alembic antigo, estrutura mista ou parcial	Reconciliação específica antes de qualquer DDL	As migrações recusam linhagens incompatíveis; `stamp` não cria objetos ausentes.

O `schema.sql` contém índices, gatilhos, período `tstzrange` e exclusão GiST que não estão integralmente representados no ORM. Alembic possui apenas revisões antigas de clínicas/usuários e pacientes/dentistas/consultas; não reproduz prontuários, evoluções, auditoria, período e concessões do SQL completo. O guia geral do banco ainda descreve oito tabelas e está desatualizado diante de `clinical_access_grants`.

Os ensaios das duas migrações ocorreram em PostgreSQL 16.15 **descartável**. Segundo o repasse SQL, nenhuma migração clínica foi aplicada ao volume persistente de revisão `lumina_s07_docker_review_20261023`. O estado atual do banco de destino não foi inventariado nesta tarefa. Antes de atualização futura, é necessário identificar host, banco e volume, comparar catálogo e contagens, preparar backup protegido, **restaurá-lo em PostgreSQL 16 separado**, ensaiar migrações e leitura, e medir bloqueios. `pg_dump -Fc` consta como plano; não há evidência de backup ou restauração efetivamente executados. Reverter uma transação de migração que falhou antes do commit não equivale a recuperação validada depois da aplicação.

5. Backend e contratos HTTP

As rotas usam `/api/v1`; `/health` é separado. O guia de Agenda e o de prontuário detalham corpos, erros e exemplos fictícios. A tabela resume as **onze rotas documentadas**, sem substituir esses contratos.

Rota	Papel e entrada principal	Comportamento e limite
`POST /appointments`	Administração/recepção; paciente, dentista, horário	Cria `SCHEDULED` na clínica da sessão; valida participantes e conflito.
`GET /appointments`	Equipe; filtros de período, dentista, paciente e situação	Dentista vê apenas a própria Agenda; limites de período usam instantes com fuso.
`GET /appointments/{id}`	Equipe autorizada; ID	Detalhe sem ampliar a visibilidade do dentista.
`PATCH /appointments/{id}`	Administração/recepção; dentista para própria situação	Edição administrativa ou transição; `COMPLETED` exige `care_occurred_at`.
`PATCH /appointments/{id}/status`	Equipe conforme ação	Mesma regra de transição do PATCH genérico.
`POST /appointments/{id}/cancel`	Administração/recepção; sem corpo	Cancela sem apagar evidências.
`GET /patients/{id}/record`	Dentista com fundamento válido	Consulta **e cria** prontuário ausente após autorização; D02 pendente.
`GET /patients/{id}/evolutions`	Dentista autorizado	Finalizadas de todos os autores e rascunhos próprios; 404 sem prontuário.
`POST /patients/evolutions`	Dentista autorizado; paciente e descrição	Cria `DRAFT`, autor da sessão; consulta vinculada deve corresponder à dupla/clínica.

Rota	Papel e entrada principal	Comportamento e limite
`PATCH /patients/evolutions/{id}/draft`	Autor com acesso vigente	Edita rascunho próprio; mutação bloqueia a linha.
`POST /patients/evolutions/{id}/finalize`	Autor com acesso vigente	Registra `FINALIZED` e `finalized_at`; não conclui a Agenda.

Administrador e recepção podem confirmar, cancelar e registrar ausência. Só o dentista responsável pode concluir a própria consulta: ela deve estar `CONFIRMED` com confirmação confiável, o horário agendado deve ter iniciado e `care_occurred_at` deve ser informado com fuso e não estar no futuro. Situação `COMPLETED` e os três campos de realização são gravados na mesma transação. Confirmação e conclusão têm evidências distintas. `SCHEDULED → COMPLETED` é recusado na API atual; o relógio ou horário agendado não comprovam realização. Finalizar uma evolução é outra ação.

O formato de erro é `{"error":{"code":"...","message":"...","details":null}}`. O guia da Agenda descreve 401, 403, 404, 409, 422 e 500; o de prontuário detalha 403 para acesso ou autoria, 404 para recurso ausente e 409 para duplicidade de evolução por consulta. Datas ISO 8601 com fuso são necessárias para `care_occurred_at`; a API interpreta alguns horários de Agenda sem fuso como UTC, por isso a interface deve enviar fuso explícito. O texto clínico não deve ser reproduzido em mensagens de validação.

6. Requisitos, regras e estado de aprovação

IDs existentes	Decisão de Paulo e proposta separada	Código encontrado	Validação e aceite
RF-017, RF-022; RN-021, RN-024, RN-025	**Aprovado:** conclusão só pelo dentista responsável após `CONFIRMED` confiável, com situação e evidência atômicas. Correção indevida segue aberta.	Serviços da Agenda aplicam a transição em ambos os PATCHs; frontend não envia `care_occurred_at`.	Ensaios técnicos históricos; falta fluxo integrado/QA.
RF-023; RN-031, RN-001, RN-017	**Aprovado em 25/10:** preparação por confirmação ativa e continuidade por atendimento comprovado; outra origem válida pode manter leitura.	Repositório consulta evidências por clínica, paciente e dentista; API verifica fundamento.	Banco de destino e versão de interface não validados; D01/D02 abertos em parte.
RF-024; RN-031, RN-032	**Aprovado:** finalizadas legíveis para dentistas autorizados; rascunhos privados ao autor. **Proposta:** estados de escrita, conteúdo mínimo, CRO histórico e retificação.	Filtro de leitura, autoria da sessão e bloqueio comum após finalização; escrita herdada aceita descrição vazia.	Testes existentes não equivalem ao aceite das propostas D03–D05.
RF-025; RN-033	Resumo por IA é proposta sujeita a D07.	Não foi identificado serviço de IA integrado neste recorte.	Sem aceite.
RF-026; RN-034	Histórico de operações é escopo proposto.	`audit_logs` no esquema, sem prova de gravação e leitura funcional.	Sem aceite.

As revisões de RN-031, RF-023, RF-024 e seus catálogos constam dos **oito arquivos locais não commitados** em 29/10. Suas descrições de “implementação pendente” refletem a etapa da decisão de 25/10 e precisam ser reconciliadas com os commits técnicos de 26/10 antes de publicação documental. A política de 24/10, que preservava leitura somente por confirmação mesmo após cancelamento sem atendimento, é **histórico substituído**, não regra atual. A revogação administrativa individual continua melhoria futura.

7. Segurança, transações e concorrência

A verificação de acesso exige usuário ativo, perfil de dentista, cadastro profissional com CRO/UF, clínica ativa, paciente da mesma clínica e ao menos uma origem assistencial válida. A API autoriza antes de expor observações gerais e texto clínico. Finalizadas são compartilhadas entre dentistas autorizados; rascunhos são retornados apenas ao autor, que também deve manter fundamento de acesso. Administrador, recepção e outra clínica não devem obter conteúdo clínico por essas rotas. O estado de paciente inativo ainda depende de D06.

Na Agenda, confirmação e concessão, ou conclusão e realização, compartilham transação; falha deve reverter ambas. A criação de rascunho e eventual abertura implícita do prontuário também compartilham transação. O código usa bloqueios de linha em operações específicas e a unicidade por consulta impede duas evoluções ligadas ao mesmo agendamento. Essas medidas cobrem disputas delimitadas, sem garantir toda a autorização durante espera concorrente.

Lacuna 1 — autorização após espera: em records/service.py, a edição/finalização verifica fundamento e autoria **antes** de aguardar o bloqueio da evolução. Depois do bloqueio, confere referência, autor e estado, mas não revalida o fundamento assistencial. Se ele se perder durante a espera, a mutação pode prosseguir. O teste PostgreSQL relatado cobre cancelamento durante espera na **criação** de rascunho, não essa edição/finalização.

Lacuna 2 — sobrescrita de rascunho: duas edições do mesmo `DRAFT` são serializadas pelo bloqueio, mas não há versão nem condição de atualização que rejeite um formulário antigo. A segunda edição pode substituir silenciosamente valores da primeira. O teste existente de espera contra finalização evita editar `FINALIZED`; não comprova resolução de conflito entre duas edições de rascunho. Ambas as lacunas exigem decisão de tratamento, correção e ensaio HTTP PostgreSQL antes do aceite.

8. Testes e evidências disponíveis

Leitura da tabela: "resultado" é o que a fonte registra. Este relatório não executou testes, banco ou Docker. Os 82 testes SQLite e cinco PostgreSQL são os **mesmos resultados** presentes nos relatos de DBA/backend e no repasse técnico; não foram somados nem repetidos.

Fonte	Data	Commit/versão	Ambiente	Comando ou roteiro	Resultado registrado	Limite
Registro Docker da Sprint 7	23/10	Integração anterior a `01a0157`	Compose isolado, PostgreSQL	Build, health, login, seed e reinício	Três serviços saudáveis; 50 pacientes fictícios recuperados	Não valida evidências clínicas nem HEAD atual.
Repasse SQL de 24/10	24/10	Base `698a3bd` + primeira migração	Dois PostgreSQL 16.15 descartáveis	Banco novo e atualização; seed, exercício, verificação e reinício	Integridade, duplicidade, concorrência e reaplicação recusada	Política antiga; ensaio estrutural, sem HTTP.
Repasse SQL de 25/10	25/10	SQL que entrou em `be7c2f2` no dia 26	Três PostgreSQL 16.15 descartáveis	`run_20261025_clinical_access_pg16.sh`: novo, upgrade e base	Três caminhos passaram; legado preservado; simulações de fundamento, rollback e concorrência	Consulta SQL simulada; não é migração do volume persistente.
Repasse da API	26/10	Etapa `ef15a63`	SQLite em memória	`pytest -q` com variáveis de teste e plugins desativados, conforme repasse	82 passaram, 5 pulados	Histórico; SQLite não valida GiST e concorrência PG.
Repasse da API	26/10	Etapa `ef15a63`	PostgreSQL 16.15 descartável	`bash tests/run_clinical_pg16.sh` em `backend/`	5 passaram; fluxo HTTP, reinício, concorrência delimitada e rollback	Banco novo; não cobre lacunas 1 e 2 nem migração do destino.
Guias HTTP e código	28–29/10	HEAD `2f7dbfe`	Inspeção estática local	Leitura de rotas, serviços, frontend, SQL e Git	Onze rotas documentadas; divergência de frontend e lacunas identificadas	Não constitui execução, QA ou demonstração da interface.

Não foi encontrado no repositório relatório de aceite final de João, captura da versão integrada atual, comprovante de migração clínica do banco persistente, backup restaurado ou evidência de uso do HEAD em imagem Docker. Testes individuais de SQL, serviço, HTTP e interface não são intercambiáveis. Saídas e capturas, se incorporadas depois, devem conter commit, data, ambiente, dados fictícios e resultado esperado/obtido.

9. Integração pendente e relação com o relatório cumulativo

O frontend ainda envia só `{status}` ao atualizar a situação. Em `App.tsx`, a interface oferece `COMPLETED` ao administrador e a partir de `SCHEDULED`, possibilidades recusadas pela API atual. A conclusão precisa coletar `care_occurred_at` com fuso para o dentista responsável e tratar respostas 403/409/422. `recordsApi.getOrOpen` chama o GET que cria prontuário; a separação entre consulta e abertura depende de D02 e mudança coordenada da interface. Botões não substituem a validação no servidor.

Decisões clínicas: D01 está parcialmente fechada pela política de preparação/continuidade, mas legado, confirmação indevida, troca excepcional de participantes e migração do destino seguem pendentes. D02 (abertura) permanece aberta. D03 fecha apenas conclusão e autoria do registro de realização; correção indevida, estados de escrita da evolução e atendimento avulso continuam abertos. D04 (conteúdo mínimo e autoria de terceiro), D05 (CRO histórico e retificação), D06 (paciente inativo) e D07 (IA) não foram aprovadas por extensão. A versão SQL/ORM e o código precisam ser avaliados contra as revisões locais, sem atribuí-las ao HEAD publicado.

Item do plano cumulativo	Classificação neste recorte	Evidência ou falta
Dashboard	Parcial no código	Existem contagens de pacientes/equipe; não foram identificados indicadores de consultas por situação e período nem aceite da interface.
Relatórios administrativos	Planejados	Não há demonstração de relatório de consultas no material verificado.
Pesquisa	Parcial no código	Há busca de pacientes na interface; pesquisa integrada no recorte acordado da Sprint 7 não foi validada.
Filtros da Agenda	Implementados no código	Período, dentista e situação aparecem na interface/API; resultado combinado na versão atual não foi aceito por João.
Exportação CSV	Planejada	Não foi encontrada implementação ou decisão formal de inaplicabilidade neste recorte.
Histórico de operações	Planejado	`audit_logs` existe no esquema, sem fluxo funcional demonstrado.
IA/OpenCL	Proposta/pesquisa	Sem integração ou medição comprovada por este recorte.

O repositório contém planos das Sprints 4 e 5 e planejamento da Sprint 7, mas **não contém** o PDF geral cumulativo nem os relatórios finais e evidências completas de todas as Sprints. Este documento técnico pode alimentar a seção da Sprint 7; Cassiano ainda precisa do arquivo editável/PDF cumulativo atual, seções anteriores aprovadas, numeração de figuras, capturas, referências e resultados finais de João. Não se inventam capítulos prévios, aceite da Pré-Banca, prorrogação acadêmica ou comprovante de envio.

10. Pendências, espaço de QA e conclusão

Prioridade	Impacto	Responsável proposto	Próxima ação	Evidência necessária
Alta	Autorização pode se perder durante espera da edição/finalização	Backend e João	Revalidar fundamento após bloqueio com ordem segura	Teste concorrente HTTP PostgreSQL, acesso negado após perda do vínculo.
Alta	Duas edições podem sobrescrever rascunho	Backend, Paulo e João	Decidir e implementar controle de conflito	Duas sessões com versão antiga e resposta explícita.
Alta	Interface diverge do contrato de conclusão	Caio/Otávio e João	Enviar horário com fuso; corrigir ações por perfil/estado	Capturas e roteiro API–interface no mesmo commit/imagem.
Alta	Banco existente não foi atualizado/validado	DBA e responsável pelo ambiente	Inventário, backup protegido, restauração ensaiada e plano de migração	Catálogo antes/depois, contagens, locks, leitura da API na restauração.
Alta	Aceite integrado inexistente	João, com Paulo	Testar versão efetivamente executada	Commit, IDs de imagem, banco, casos por perfil/clínica, resultados e defeitos.
Média	Abertura do prontuário, escrita e legado ainda sem fechamento	Paulo, backend, DBA e frontend	Decidir D01–D06 nos pontos abertos	Regra aprovada, contrato, implementação e testes vinculados.
Média	Alembic e guia geral divergem do SQL completo	DBA e Tech Lead	Escolher estratégia de evolução e atualizar inventário	Comparação de esquema e documentação coerente.
Média	PDF cumulativo e itens não clínicos sem evidências completas	Cassiano, equipe e Paulo	Integrar este recorte, obter arquivos anteriores e provas	Arquivo geral, figuras, links, evidências e revisão da equipe.

10.1 Espaço reservado — resultados finais de João

Ainda não recebido neste repositório em 29/10/2026. Preencher somente após execução e conferência; nenhum campo abaixo representa aprovação.

Campo a registrar	Resultado/evidência a anexar por João
Commit, imagem e banco efetivamente testados	Pendente.
Roteiro de perfis e clínicas, com dados fictícios	Pendente.
Confirmação, conclusão e `care_occurred_at` pela interface	Pendente.
Leitura de finalizadas, privacidade de rascunhos e erros	Pendente.
Concorrência, rollback, reinício e banco existente	Pendente.
Defeitos restantes, aceite e data da revisão	Pendente.

10.2 Conclusão

API pronta para novos ensaios técnicos delimitados: o código e o esquema versionados permitem preparar banco PostgreSQL 16 exclusivo e descartável e repetir casos SQL/HTTP. **Interface e banco existente não estão prontos para aceite com base nas evidências disponíveis:** falta ajustar o frontend, identificar a imagem executada, inventariar o destino e ensaiar recuperação/migração. **Aceite final da Sprint 7 permanece pendente** de João, das decisões clínicas abertas e dos demais itens do plano acadêmico. Os responsáveis desta seção são propostas de encaminhamento, não compromissos confirmados de integrantes.

11. Fontes e rastreabilidade

ID	Fonte	Uso e limite
F1	Relatório técnico consolidado do DBA, entregue por Paulo em 29/10/2026 nesta conversa	Síntese de estrutura, migrações e ensaios; relato de autor, confrontado com SQL e repasse versionado.
F2	Relatório técnico consolidado de backend, entregue por Paulo em 29/10/2026 nesta conversa	Síntese de rotas, segurança, integração e testes; relato de autor, confrontado com código.
F3	Plano da Sprint 7 e registro Docker	Metas acadêmicas e ensaio histórico de 23/10, não aceite do HEAD.
F4	Repasse SQL, esquema e migrações	Estrutura e resultados relatados em destinos descartáveis.
F5	Repasse da API, guia Agenda e guia clínico	Contratos e contagens históricas de testes.
F6	Requisitos e decisões locais, RN-031, RF-023	Revisões locais de 25/10 ainda sem commit; não atribuídas ao HEAD.
F7	Git local: `git log`, `git status`, `git diff b224faa..HEAD`	Datas de commit, escopo versionado e oito alterações locais; não confirma estado remoto ao vivo.

Conferência desta elaboração: leitura documental e estática em 29/10/2026. Não houve execução de suíte, migração, backup, restauração, inventário Docker, alteração de código ou aceite de QA. Para o relatório cumulativo, Cassiano deve citar a versão de cada evidência e atualizar esta seção quando receber os resultados finais.